



# Dimension Reduction: Manifold Learning, Data Representation and A Unified Framework

COMP7404

Computational Intelligence and Machine Learning

Dong Xu

# Outline

- ✓ Manifold Learning Methods
- ✓ Dimensionality Reduction for Tensor-based Objects
- ✓ Graph Embedding: A General Framework for Dimensionality Reduction

# LLE

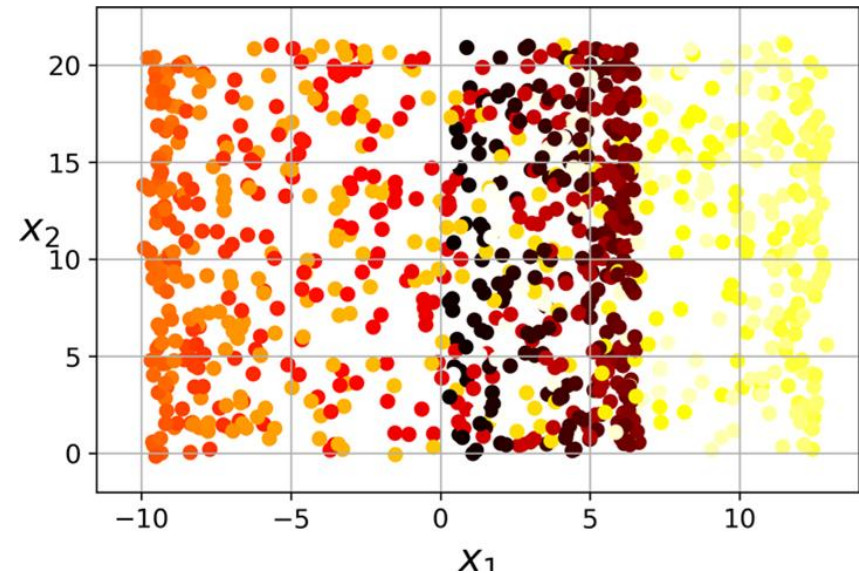
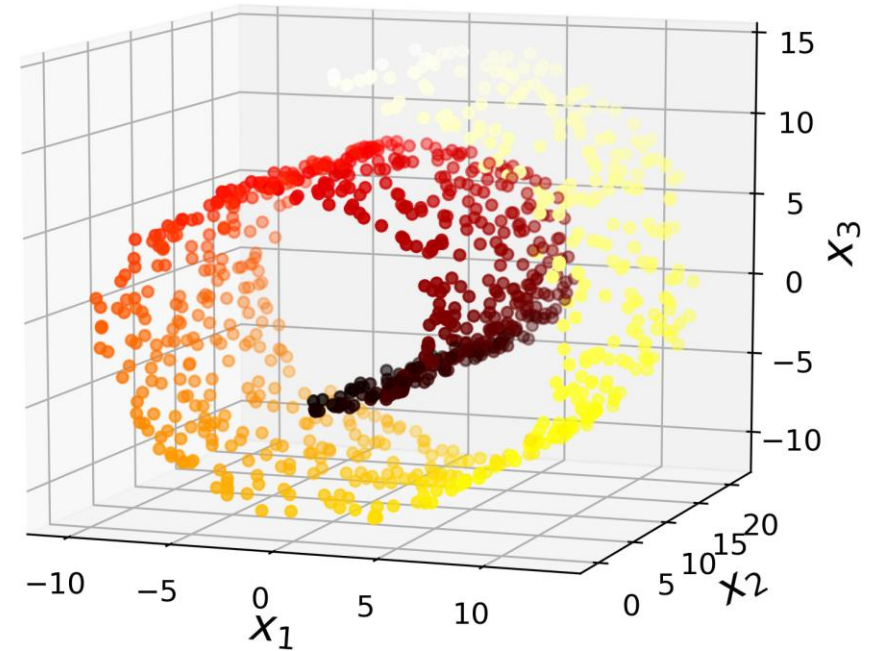
- How it works
  - Measures how each training instance linearly relates to its closest neighbors
  - Then looks for a low-dimensional representation of the training set where these local relationships are best preserved
- This approach makes it particularly good at unrolling twisted manifolds, especially when there is not too much noise

```
1 from sklearn.manifold import LocallyLinearEmbedding
2 lle = LocallyLinearEmbedding(n_components=2, n_neighbors=10)
3 X_reduced = lle.fit_transform(X)
```

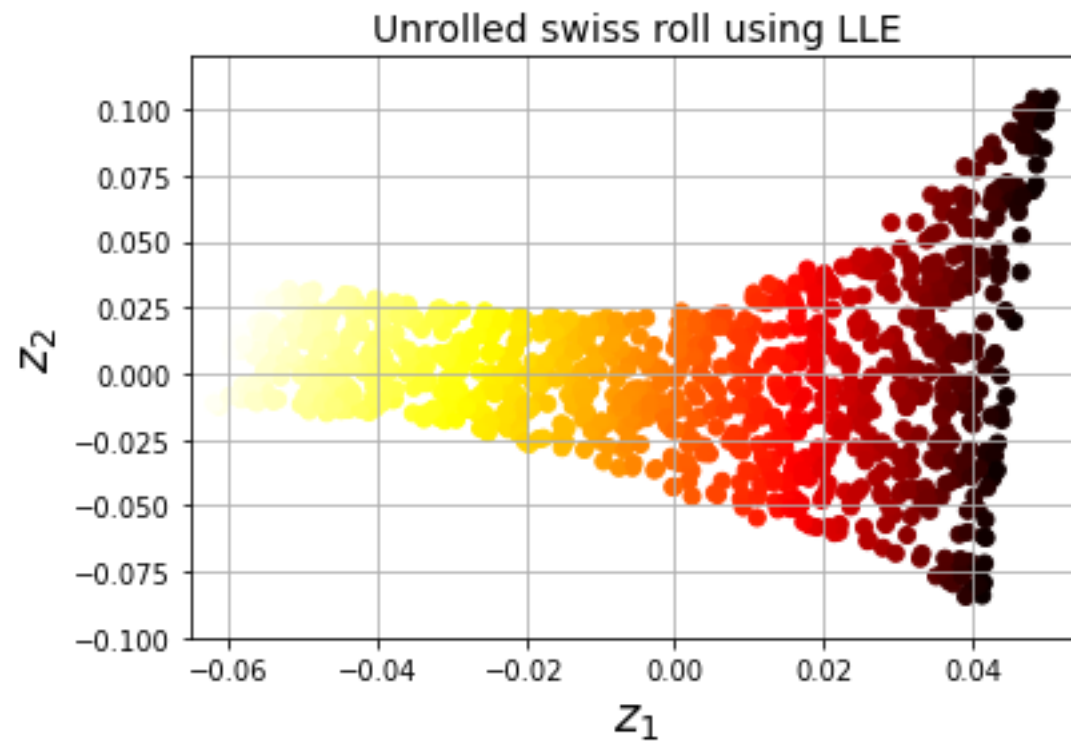
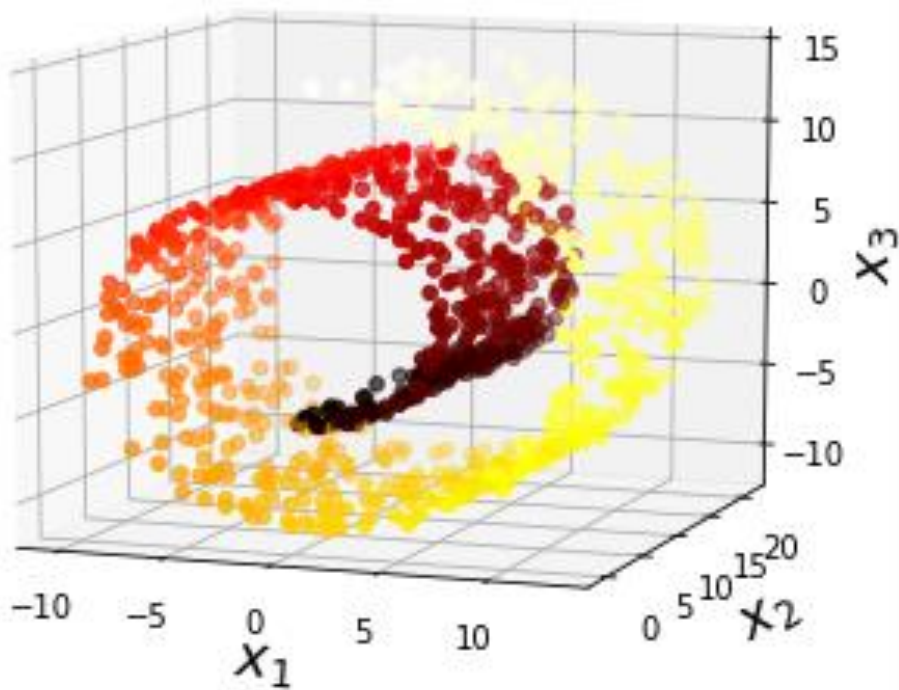
- Available [here](#) on CoLab

# When Projection Fails

- Projection is not always the best approach
- Consider the following toy dataset to illustrate this problem
  - The Swiss roll
- Simply projecting onto a plane (e.g., dropping  $x_3$ ) would squash different layers



# LLE: Unrolling the Swiss roll



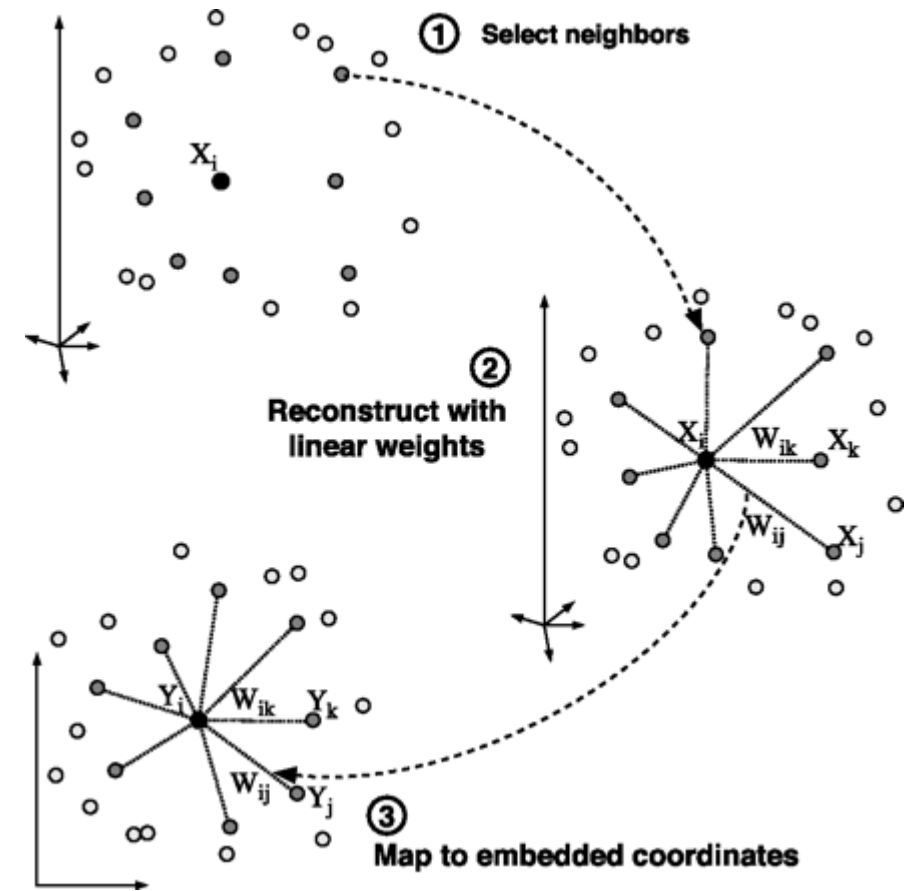
# Locally Linear Embedding (LLE)

- History: S. Roweis and L. Saul, Science, 2000
- Procedure
  1. Identify the neighbors of each data point
  2. Compute weights that best linearly reconstruct the point from its neighbors

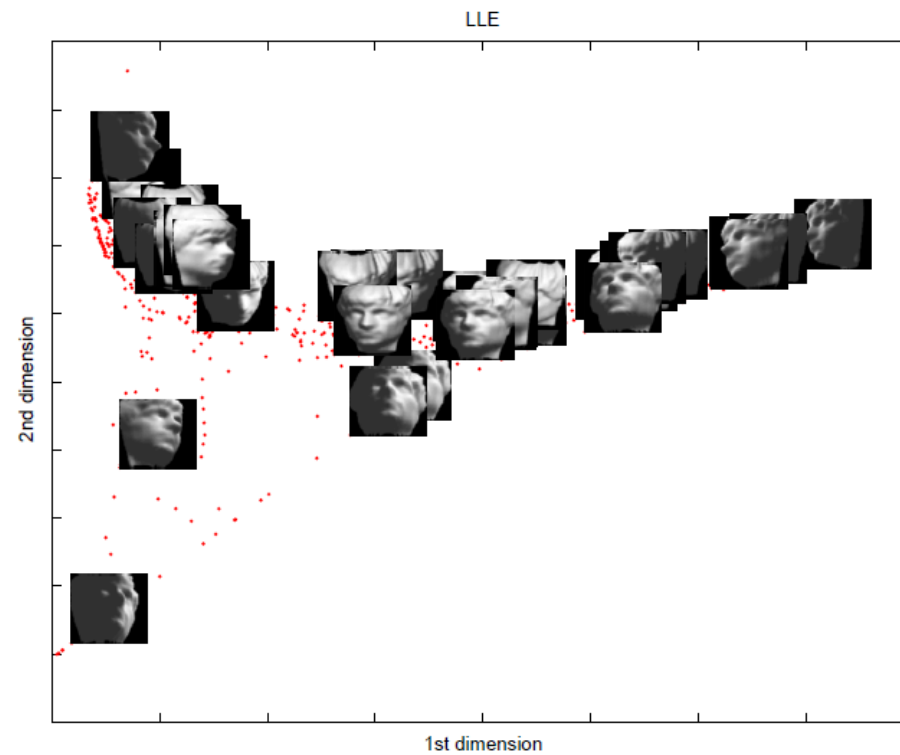
$$\min_{\mathbf{w}} \sum_{i=1}^N \left\| \mathbf{x}_i - \sum_{j=1}^k w_{ij} \mathbf{x}_{N_i(j)} \right\|^2$$

3. Find the low-dimensional embedding vector which is best reconstructed by the weights determined in Step 2

$$\min_Y \sum_{i=1}^N \left\| \mathbf{y}_i - \sum_{j=1}^k w_{ij} \mathbf{y}_{N_i(j)} \right\|^2 \iff \min_Y \text{tr}(\mathbf{Y}^\top \mathbf{Y} \mathbf{L}) \quad \text{Centering Y with unit variance}$$



# LLE Example



# Laplacian Eigenmaps (LEM)

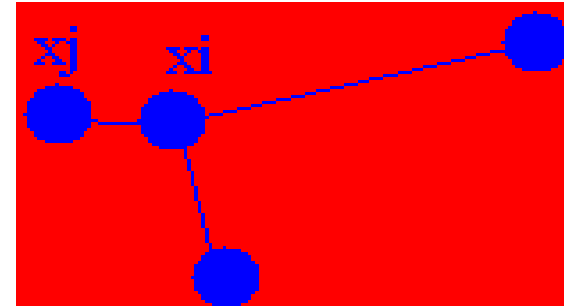
- History: M. Belkin and P. Niyogi, 2003
- Similar to locally linear embedding
- Different in weights setting and objective function

- Weights  $W_{ij} = \begin{cases} 1 & i, j \text{ are connected} \\ \exp\left(\frac{-\|x_i - x_j\|^2}{s}\right) & \text{otherwise} \end{cases}$

- Objective

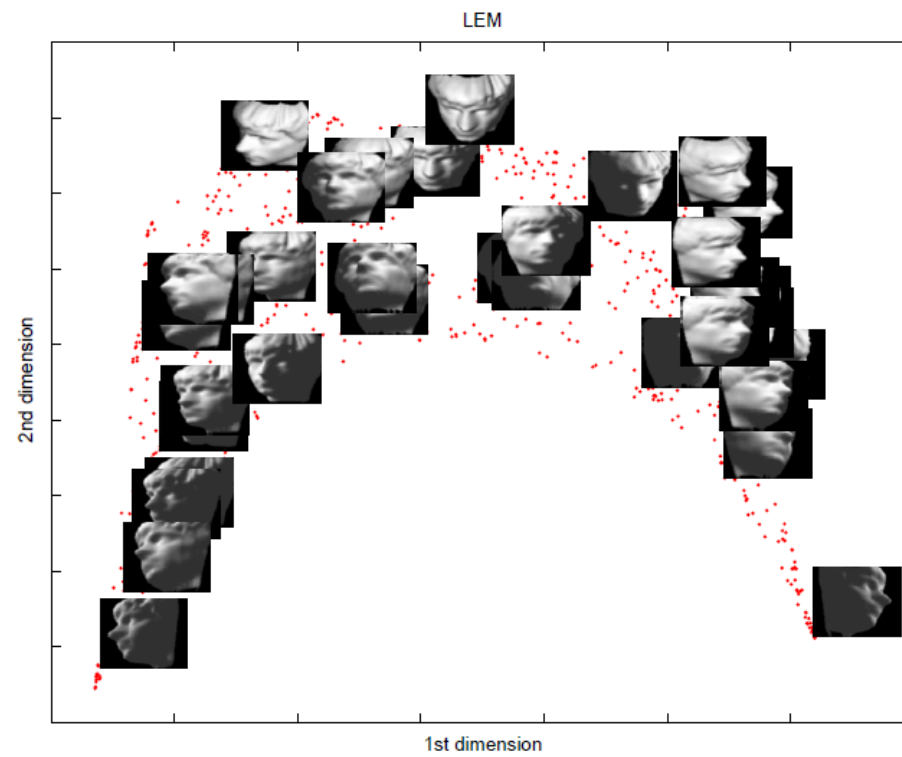
$$\min_Y \sum_{i=1}^N \sum_{j=1}^N (\mathbf{y}_i - \mathbf{y}_j) W_{ij} \iff \min_Y \text{tr}(YLY^\top)$$

where  $L = R - W$ ,  $R$  is diagonal and  $R_{ii} = \sum_{j=1}^N W_{ij}$ .





# LEM Example



# Multidimensional Scaling (MDS)

- History: T. Cox and M. Cox, 2001
- Attempts to preserve pairwise distances
- Different formulation of PCA, but yields similar result form

$$\min_Y \sum_{i=1}^N \sum_{j=1}^N (d_{ij}^{(X)} - d_{ij}^{(Y)})^2$$

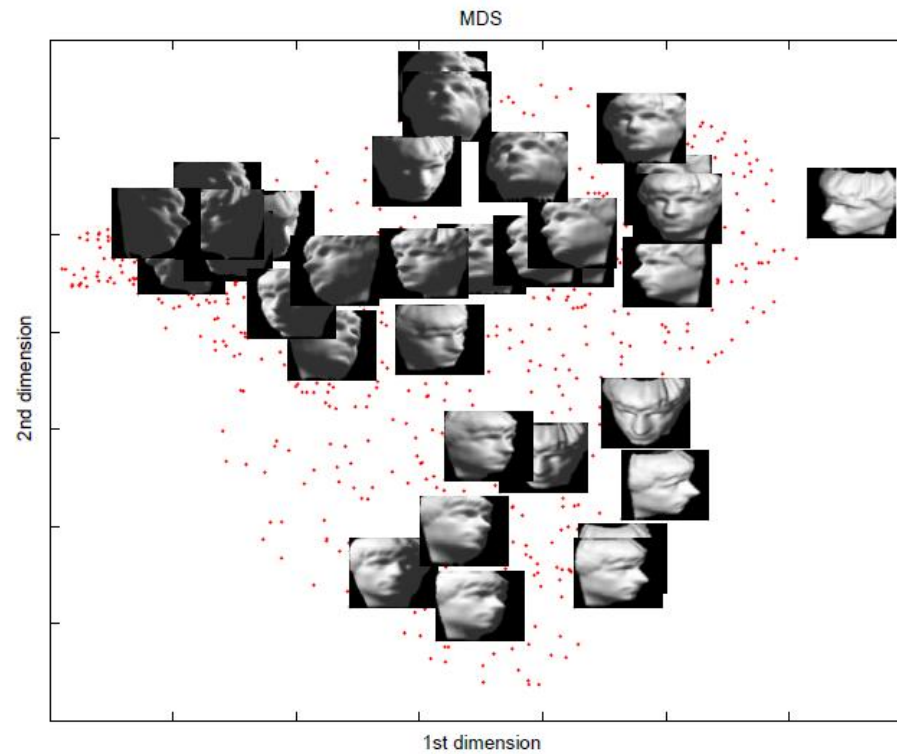
where  $d_{ij}^{(X)} = \|x_i - x_j\|^2$  and  $d_{ij}^{(Y)} = \|y_i - y_j\|^2$ .

- Transformation

$$X^\top X = -\frac{1}{2} H D^{(X)} H \quad \text{where } H = I - \frac{1}{N} \mathbf{1}\mathbf{1}^\top.$$

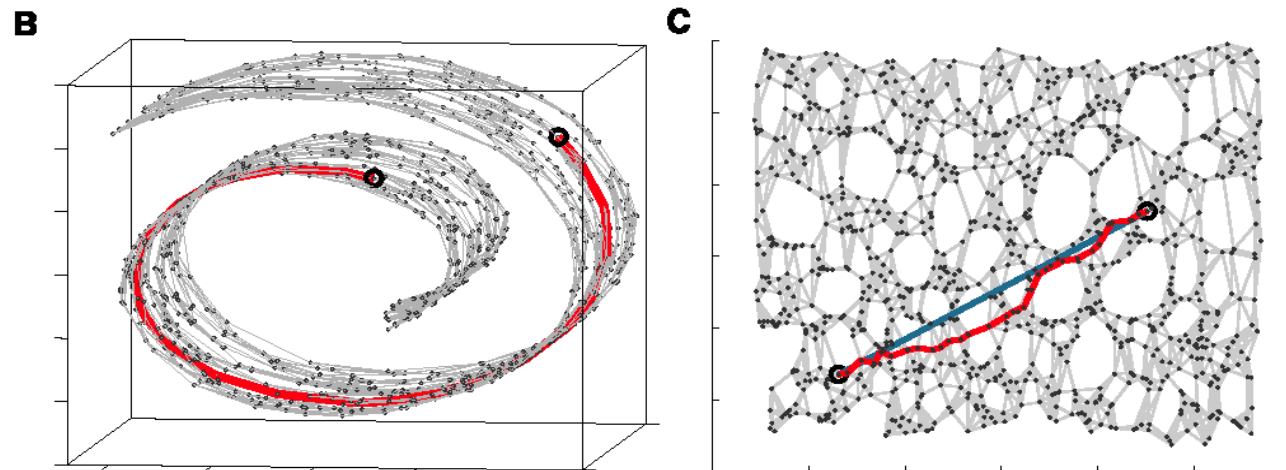
$$\min_Y \sum_{i=1}^N \sum_{j=1}^N (x_i^\top x_j - y_i^\top y_j)^2$$

# MDS Example



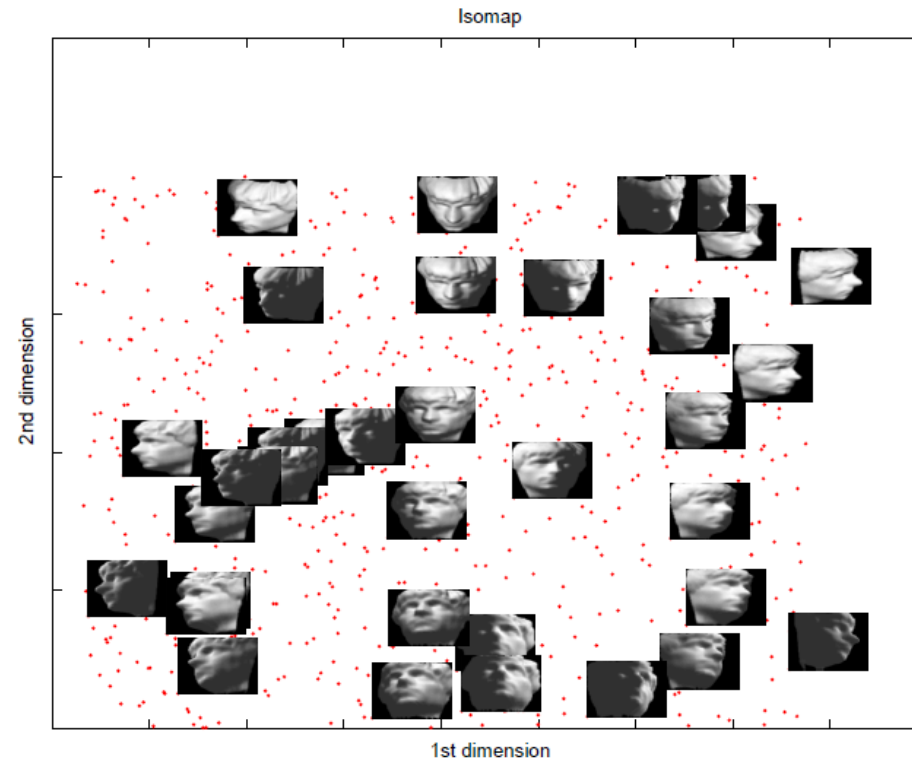
# Isomap

- History: J. Tenenbaum et al, Science 1998
- A nonlinear generalization of classical MDS
- Perform MDS, not in the original space, but in the geodesic space
- Procedure-similar to LLE
  1. Find neighbors of each data point
  2. Compute geodesic pairwise distances (e.g., shortest path distance) between all points
  3. Embed the data via MDS



Example: 3D space to 2D space

# Isomap Example



# Outline

- ✓ Manifold Learning Methods
- ✓ Dimensionality Reduction for Tensor-based Objects
- ✓ Graph Embedding: A General Framework for Dimensionality Reduction

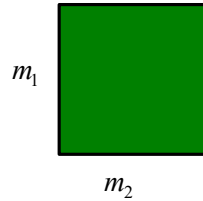
# What is Tensor?

Tensors are arrays of numbers which transform in certain ways under coordinate transformations.



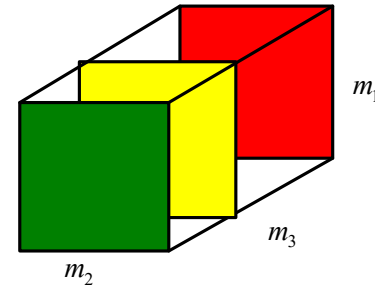
$$x \in \mathbb{R}^{m_1}$$

*Vector*



$$X \in \mathbb{R}^{m_1 \times m_2}$$

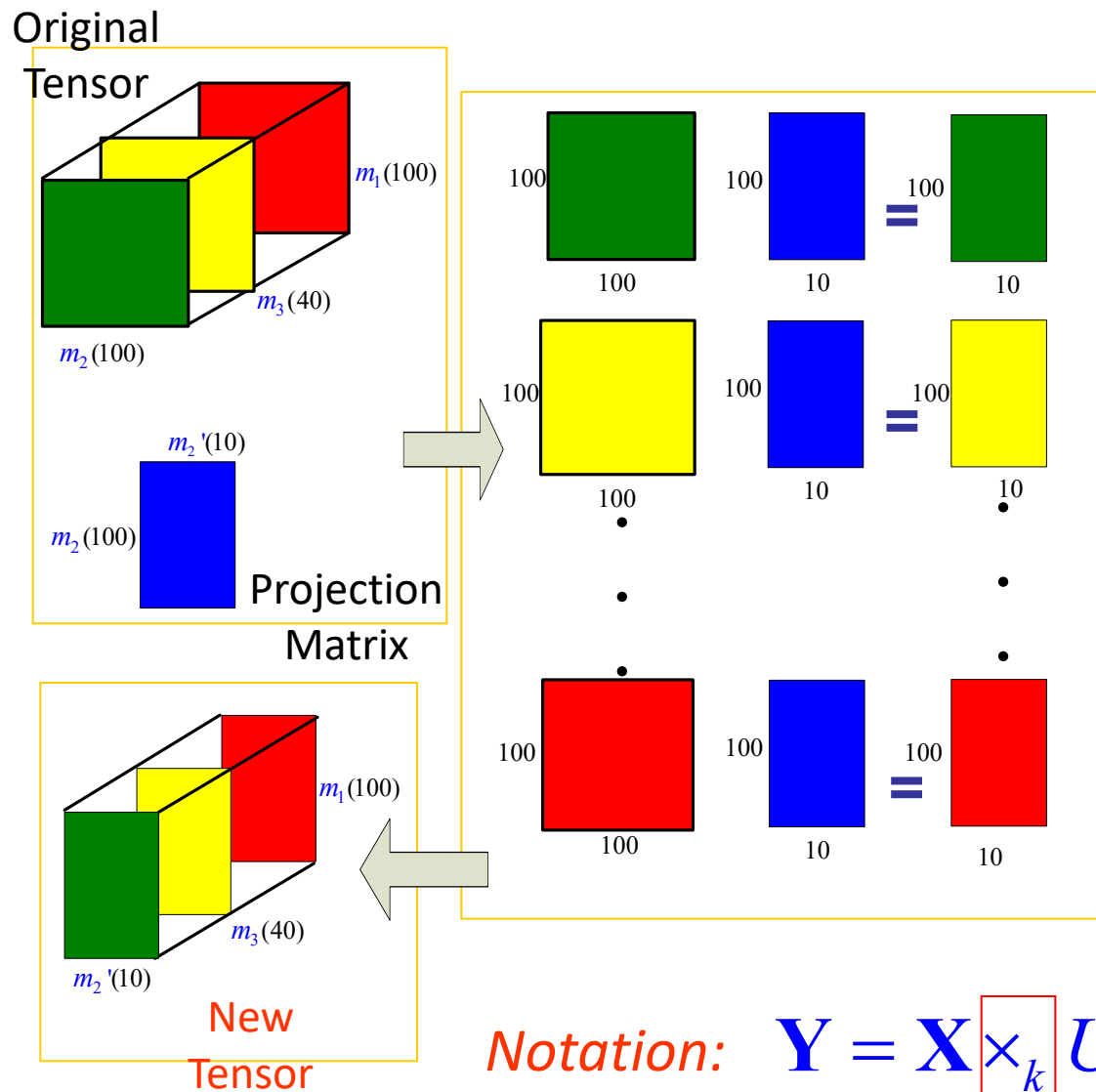
*Matrix*



$$\mathbf{X} \in \mathbb{R}^{m_1 \times m_2 \times m_3}$$

*3rd-order Tensor*

# Definition of Mode- $k$ Product



**Projection:**

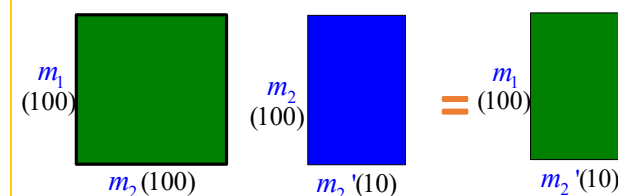
high-dimensional space  
 -> low-dimensional space

**Reconstruction:**

low-dimensional space  
 -> high-dimensional space

Product for two Matrices

$$\mathbf{Y} = \mathbf{X}\mathbf{U} \quad Y_{ij} = \sum_{k=1}^{m_2} X_{ik} U_{kj}$$



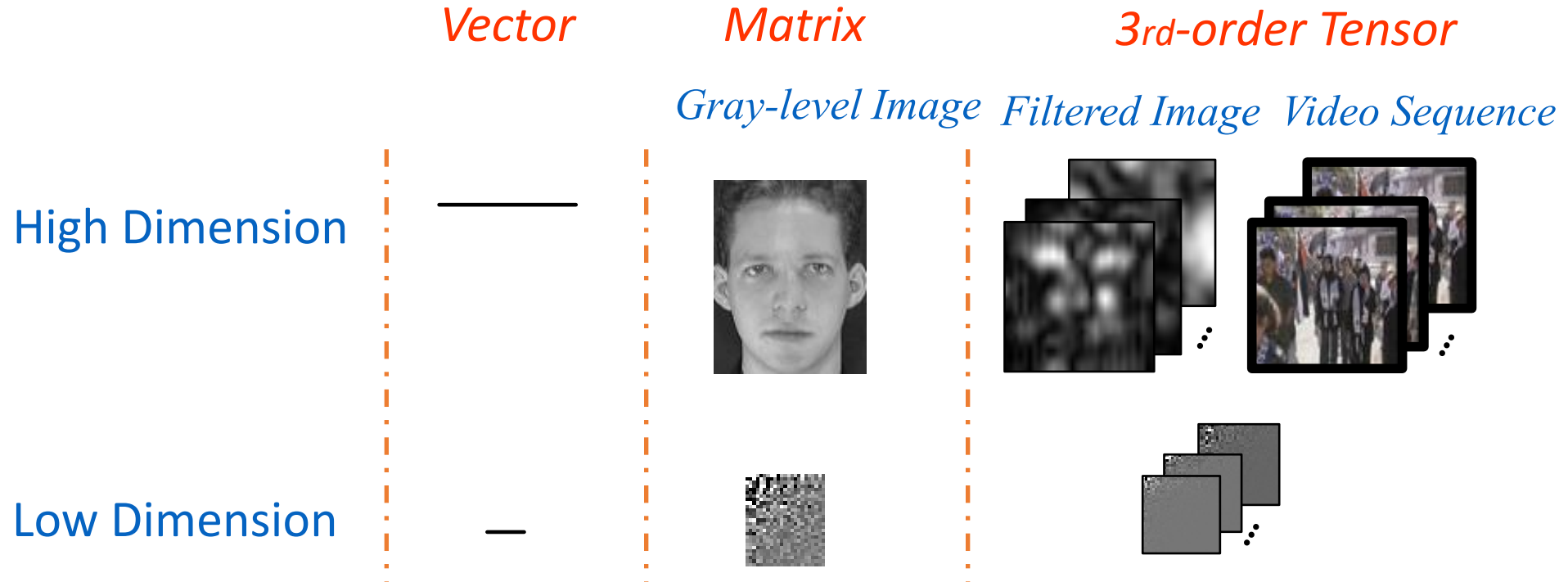
Original  
Matrix

Projection  
Matrix

New  
Matrix



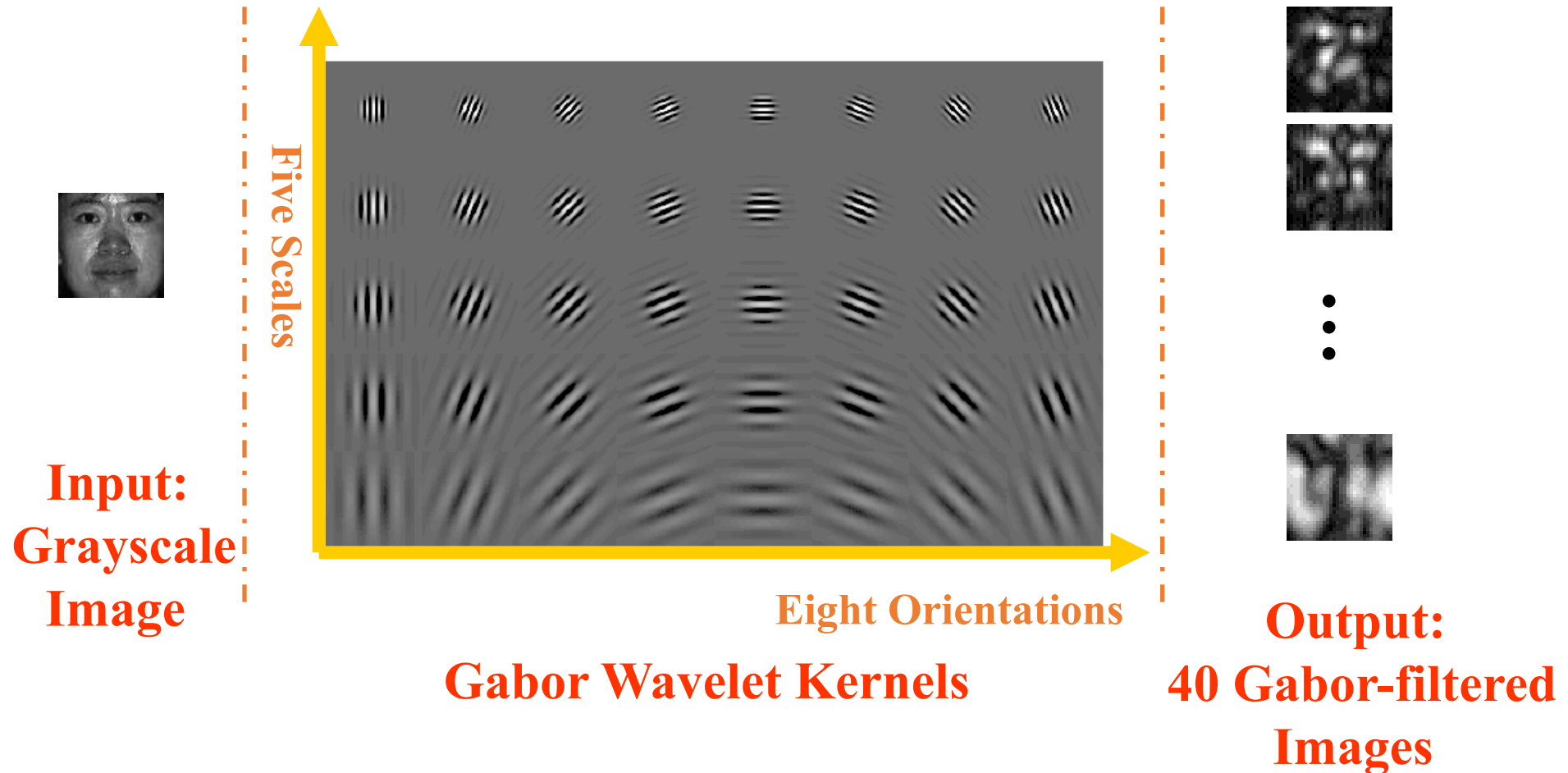
# Data Representation in Dimensionality Reduction



|          |                                                                |                             |                                                                      |
|----------|----------------------------------------------------------------|-----------------------------|----------------------------------------------------------------------|
|          | <i>PCA, LDA</i>                                                | <i>Rank-1</i>               |                                                                      |
|          |                                                                | <i>Decomposition, 2001</i>  |                                                                      |
| Examples | <i>Tensorface, 2002</i><br>M. Vasilescu and<br>D. Terzopoulos, | A. Shashua<br>and A. Levin, | <i>Our Work</i><br><i>Xu et al., 2005</i><br><i>Yan et al., 2005</i> |

# What is Gabor Features?

Gabor features can improve recognition performance in comparison to grayscale features. *Chengjun Liu T-IP, 2002*



# Why Represent Objects as *Tensors* instead of *Vectors*?

## ➤ *Natural Representation*

*Gray-level Images (2D structure)*

*Videos (3D structure)*

*Gabor-filtered Images (3D structure)*



## ➤ *Enhance Learnability in Real Application*

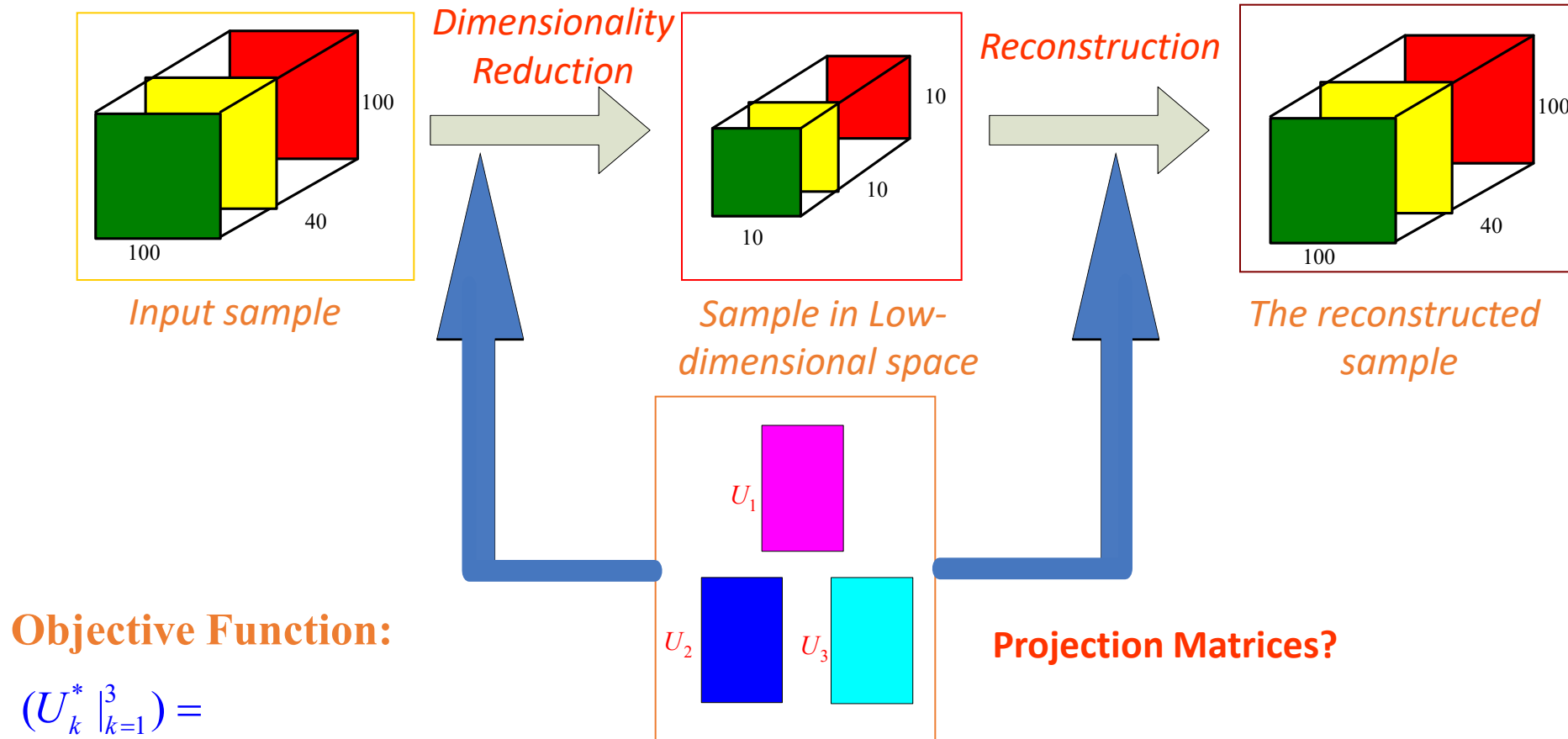
*Curse of Dimensionality (*Gabor-filtered image*:  $100 \times 100 \times 40$  -> *Vector*: 400,000)*

*Small sample size problem (less than 5,000 images in common face databases)*

## ➤ *Reduce Computation Cost*

# Concurrent Subspace Analysis as an Example

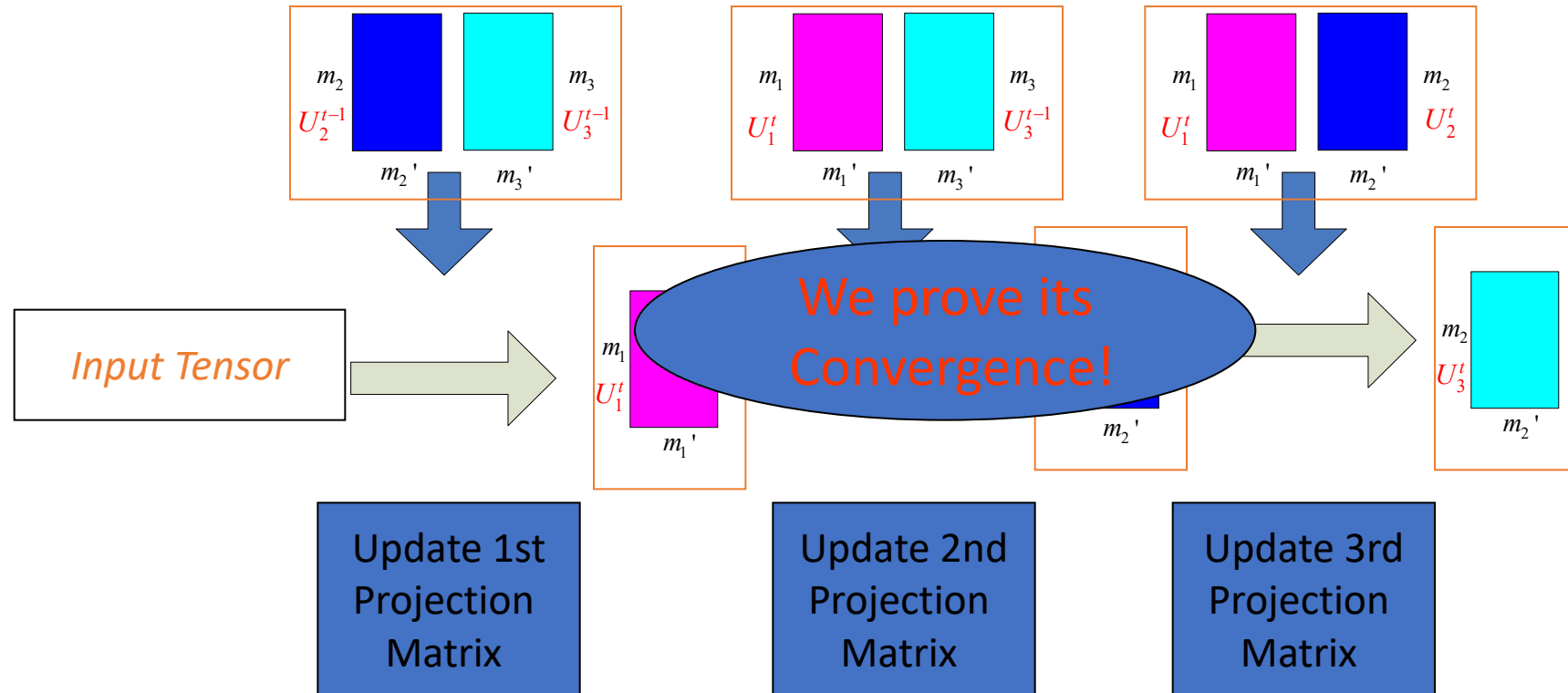
(Criterion: *Optimal Reconstruction*)



**Objective Function:**

$$(U_k^* |_{k=1}^3) = \arg \min_{U_k |_{k=1}^3} \sum_i \| \mathbf{X}_i \times_1 U_1 U_1' \dots \times_3 U_3 U_3' - \mathbf{X}_i \|^2$$

# Iterative Solution



## ***Variants of Rank-( $R_1, R_2, \dots, R_n$ ) Approximation***

*L. Lathauwer, B. Moor and J. Vandewalle, SIAM Journal of Matrix Analysis and Applications, 2001*

# Tensor Representation: *Advantages*

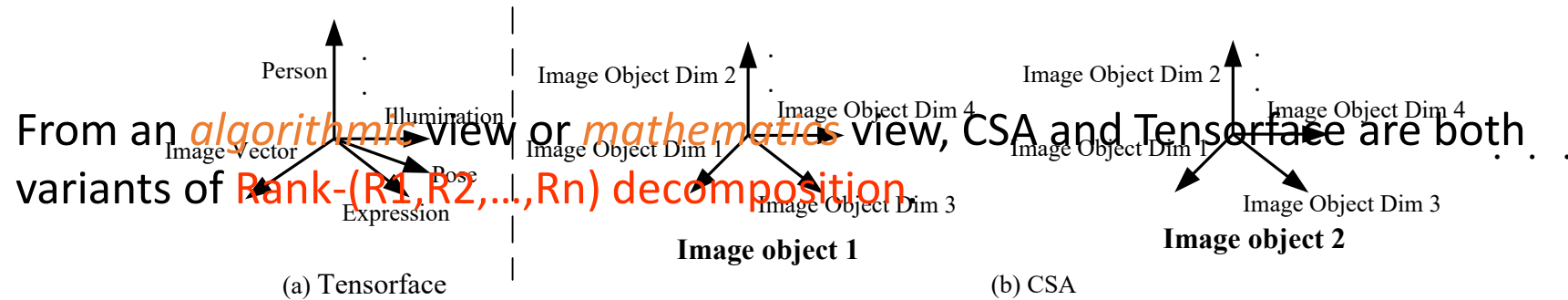
1. *Enhanced learnability*
2. *Appreciable reductions in computational costs*
3. *Large number of available projection directions*
4. *Utilize the structure information*

Correlation along the column vectors of the *mode-k* flattened matrices.

|                               | PCA                 | CSA                        |
|-------------------------------|---------------------|----------------------------|
| <i>Feature Dimension</i>      | $\prod_{k=1}^n m_k$ | $m_k$                      |
| <i>Sample Number</i>          | $N$                 | $\prod_{i \neq k} m_i * N$ |
| <i>Computation Complexity</i> | $O(m^{3n})$         | $O(n * m^{n+1})$           |

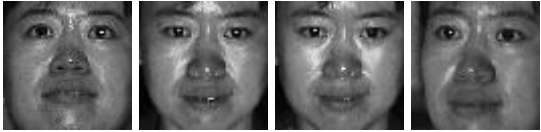
# Connection to Previous Work – *Tensorface*

(M. Vasilescu and D. Terzopoulos, 2002)



|                                                  | Tensorface                                                                       | CSA                                  |
|--------------------------------------------------|----------------------------------------------------------------------------------|--------------------------------------|
| Motivation                                       | Characterize <i>external</i> factors                                             | Characterize <i>internal</i> factors |
| Input: Gray-level Image                          | <i>Vector</i>                                                                    | <b>Matrix</b>                        |
| Input: Gabor-filtered Image<br>(Video Sequence ) | <i>Not address</i>                                                               | <b>3rd-order tensor</b>              |
| When equal to PCA                                | The number of images per person are only <i>one</i> or are <i>a prime number</i> | <b>Never</b>                         |
| Number of Images per Person for Training         | <i>Lots of images</i> per person                                                 | <b>One image</b> per person          |

# Experiments: *Database Description*

|                             | Number of Persons<br>(Images per person) | Image Size<br>(Pixels) | Example Images                                                                       |
|-----------------------------|------------------------------------------|------------------------|--------------------------------------------------------------------------------------|
| Simulated Video<br>Sequence | 60 (1)                                   | 64×64×13               |   |
| ORL database                | 40 (10)                                  | 56×46                  |   |
| CMU PIE-1 sub-<br>database  | 60 (10)                                  | 64×64                  |   |
| CMU PIE-2 sub-<br>databases | 60 (10)                                  | 64×64                  |  |



# Experiments: *Object Reconstruction (1)*

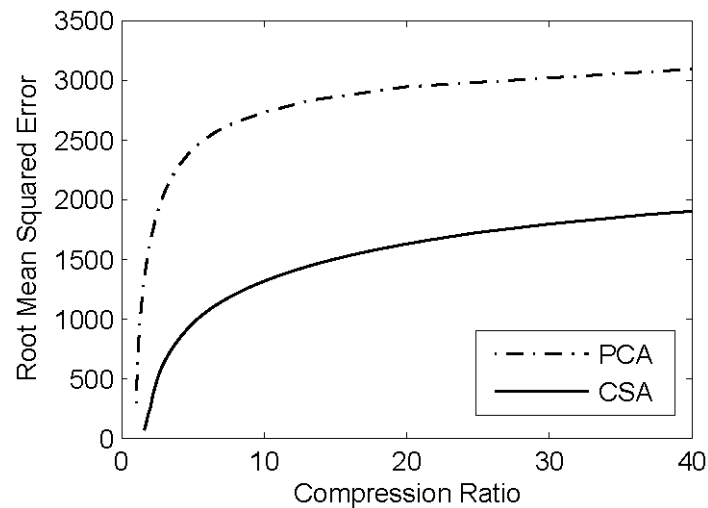
**Input:** Gabor-filtered images

ORL database

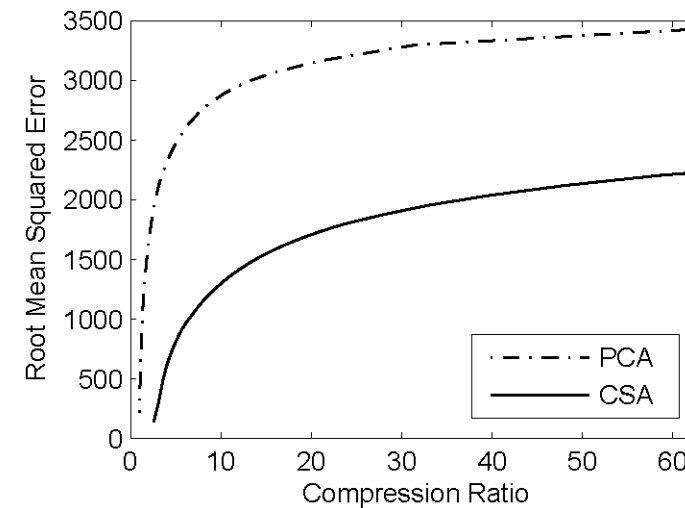
CMU PIE-1 database

*Objective Evaluation Criterion:*

Root Mean Squared Error (**RMSE**) and Compression Ratio (**CR**)



ORL database



CMU PIE-1 database

# Experiments: *Object Reconstruction (2)*

**Input:** Simulated video sequence

Original Images



Reconstructed Images from PCA



Reconstructed Images from CSA



# Experiments: *Face Recognition*

**Input:** Gray-level images and Gabor-filtered images

ORL database

CMU PIE database

| Algorithm                | CMU PIE-1    | CMU PIE-2     | ORL          |
|--------------------------|--------------|---------------|--------------|
| PCA (Gray-level feature) | 70.1%        | 28.3%         | 76.9%        |
| PCA (Gabor feature)      | 80.1%        | 42.0%         | 86.6%        |
| <b>CSA (Ours)</b>        | <b>90.5%</b> | <b>59.4 %</b> | <b>94.4%</b> |

# Summary

- This is the first work to address dimensionality reduction with a tensor representation of arbitrary order.
- Opens a new research direction.

# Tensorization - *New* Research Direction

## ➤ Our Extensions:

- 1) Supervised Learning with Rank-( $R_1, R_2, \dots R_n$ ) Decomposition (**DATER**):  
*CVPR 2005 and T-IP 2007*
- 2) Supervised Learning with Rank-1 Decomposition and Adaptive Margin (**RPAM**):  
*CVPR 2006 and T-SMC-B 2007*
- 3) Application in Human Gait Recognition (**CSA-2+DATER-2**): *T-CSVT 2006 and T-IP 2007*
- 4) Element Rearrangement: *CVPR 2007 and T-PAMI 2009*

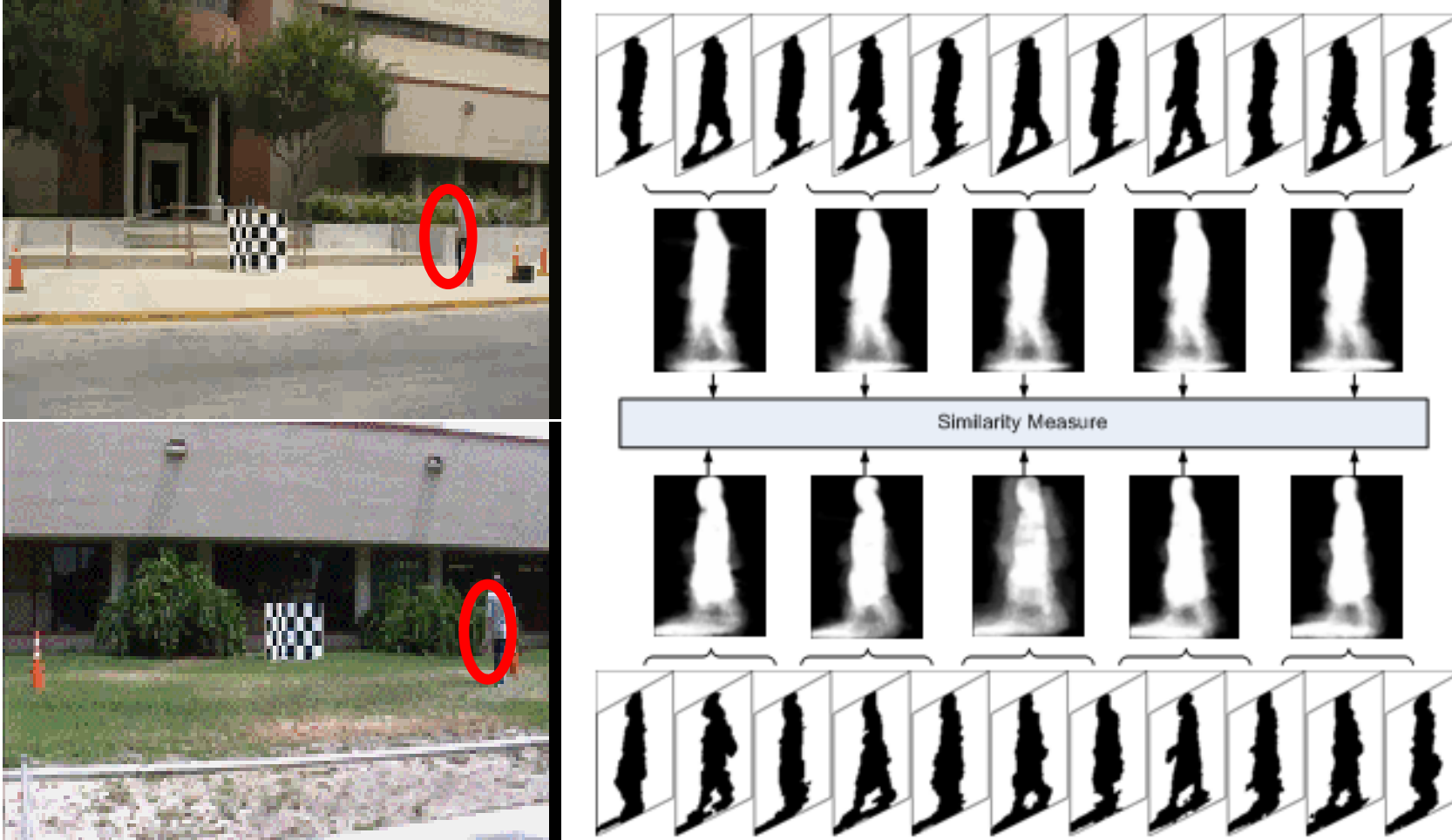
## ➤ D. Tao, S. Maybank, et al.'s Extensions :

- 1) Incremental Learning with Tensor Representation: *ACM SIGKDD 2006*
- 2) Tensorized *SVM* and *Minimax Probability Machines*: *ICDM 2005*

## ➤ G. Dai and D. Yeung's Extensions:

Tensorized *NPE* (Neighborhood Preserving Embedding), *LPP* (Locality Preserving Projections) and *LDE* (Local Discriminant Embedding): *AAAI 2006*

# Human Gait Recognition with *Matrix Representation*

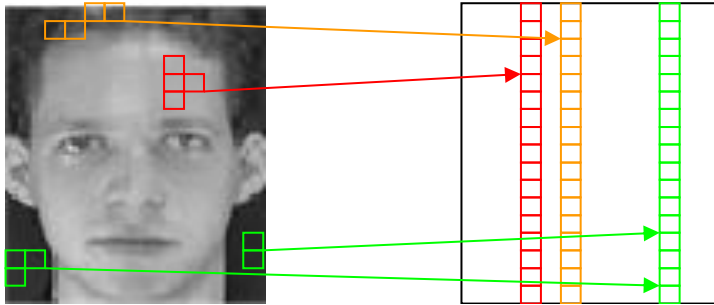


# Element Rearrangement (CVPR07 and T-PAMI09)

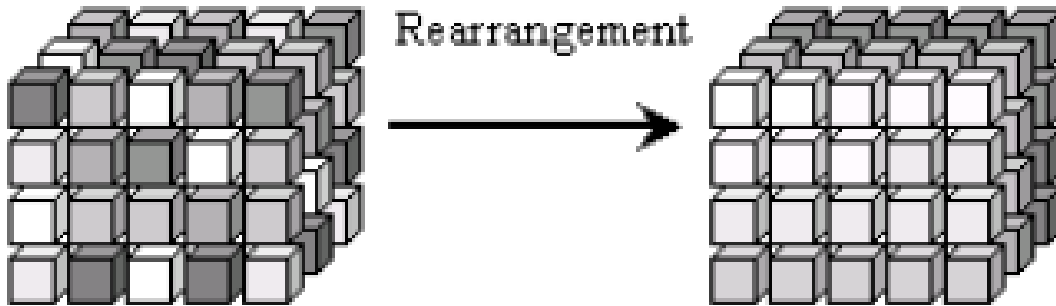
Contradiction:

- The success of tensor-based subspace learning relies on the redundancy among the unfolded vector
- The truth is that this kind of correlation/redundancy is often not strong in real data

Pixel Rearrangement



Element  
Rearrangement



Low correlation



High correlation



# Problem Definition

- The task of enhancing correlation/redundancy among 2<sup>nd</sup>-order tensor is to search for a pixel rearrangement operator  $R$ , such that

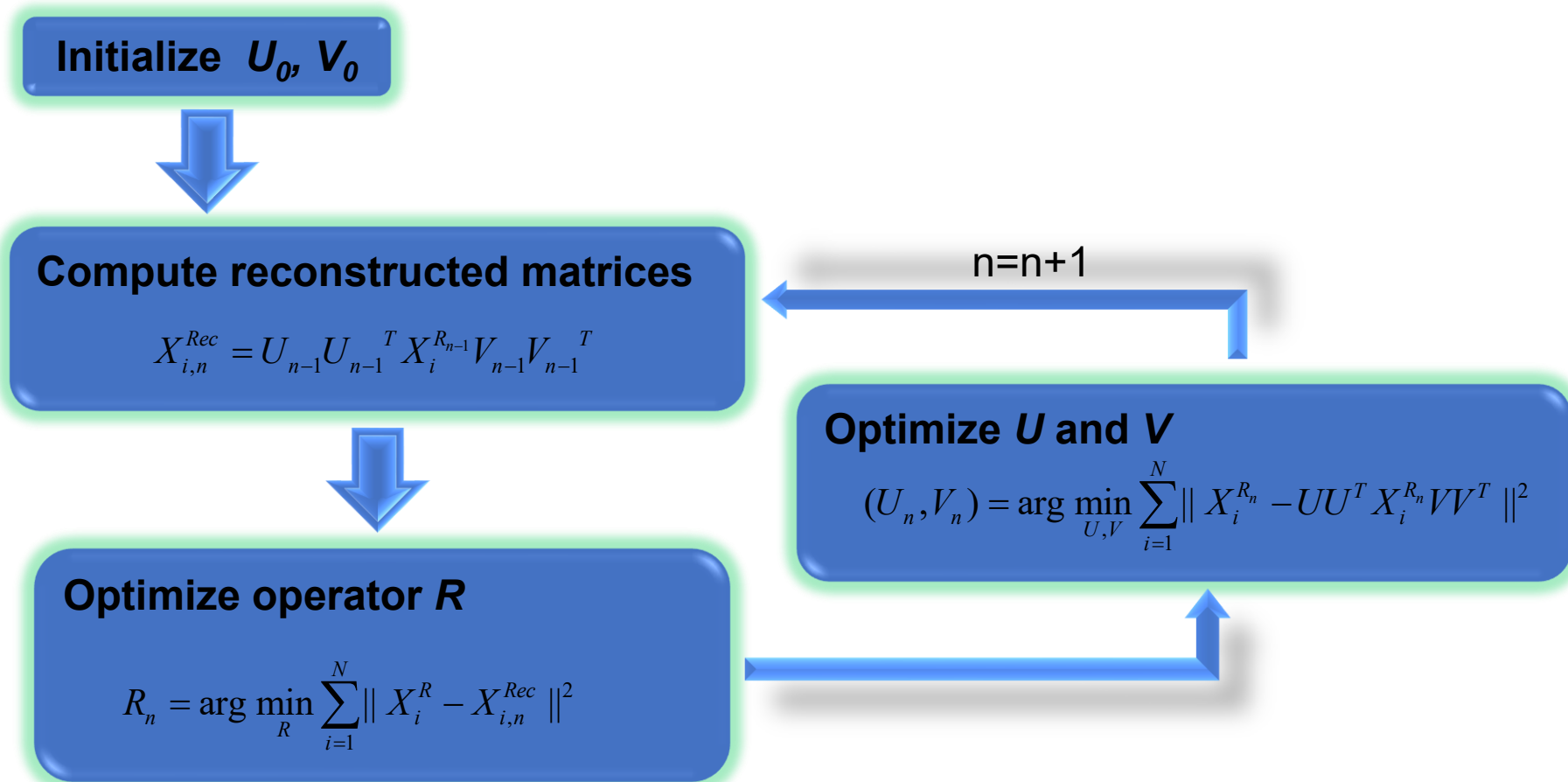
$$R^* = \arg \min_R \{ \min_{U,V} \sum_{i=1}^N \| X_i^R - UU^T X_i^R VV^T \|^2 \}$$

1.  $X_i^R$  is the rearranged matrix from sample  $X_i$
2. The column numbers of  $U$  and  $V$  are predefined

After the pixel rearrangement, we can use the rearranged tensors as input for Tensorization of graph embedding!



# Solution to Pixel Rearrangement Problem



*Note:* 
$$\sum_{i=1}^N \|X_i^{R_n} - X_{i,n}^{Rec}\|^2 \geq \sum_{i=1}^N \|X_i^{R_n} - U_{n-1} U_{n-1}^T X_i^{R_n} V_{n-1} V_{n-1}^T\|^2$$

# Step for Optimizing R

- It is Earth Mover Distance problem

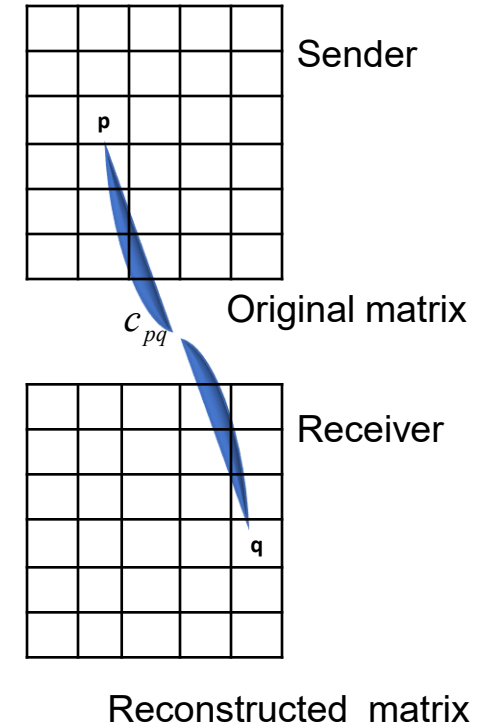
$$R^* = \arg \min_R \sum_{i=1}^N \|X_i^R - X_{i,n}^{Rec}\|^2$$



$$\min_R \sum_{p,q} c_{pq} R_{pq} \text{ st.}$$

$$1: 0 \leq R_{pq} \leq 1; \quad 2: \sum_p R_{pq} = 1; \quad 3: \sum_q R_{pq} = 1$$

$$\text{where } c_{pq} = \sum_{i=1}^N |X_i(p) - X_{i,n}^{Rec}(q)|^2$$



1. Linear programming problem has integer solution.
2. We constrain the rearrangement within local neighborhood for speedup.

# Convergence Speed

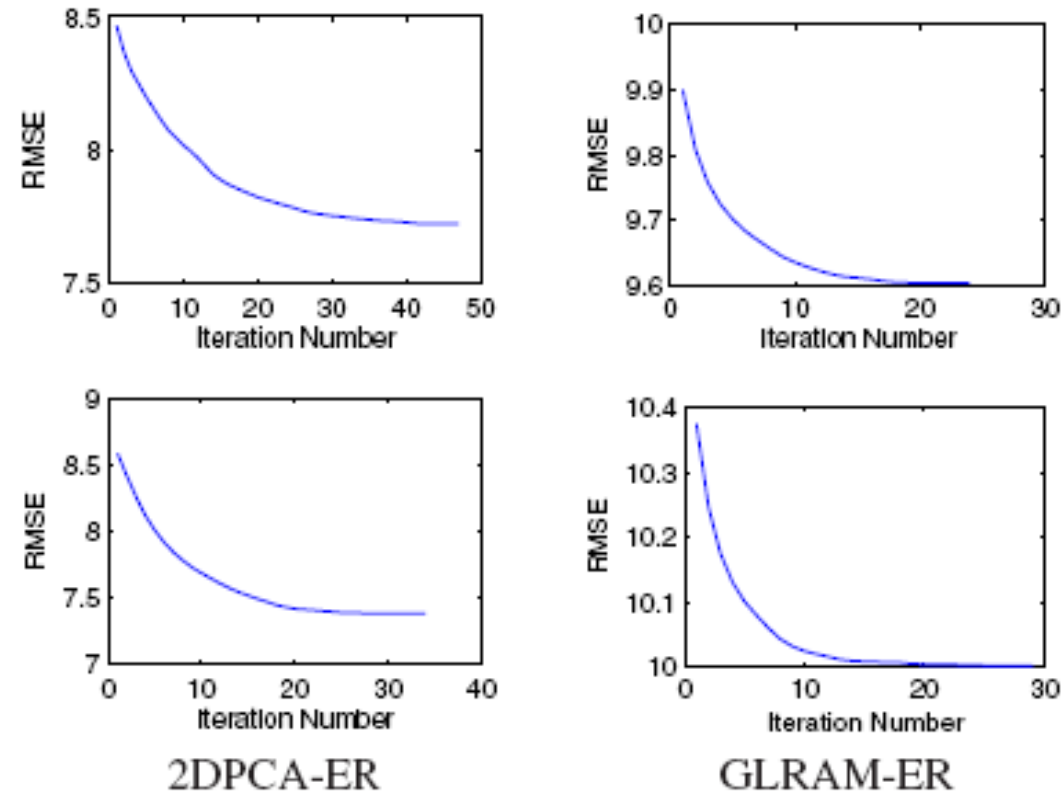


Figure 2. Algorithm convergence with element rearrangement. Root Mean Squared Error (RMSE) vs. number of iterations on the CMU PIE (top) and FERET (bottom) databases.

# Rearrangement Results

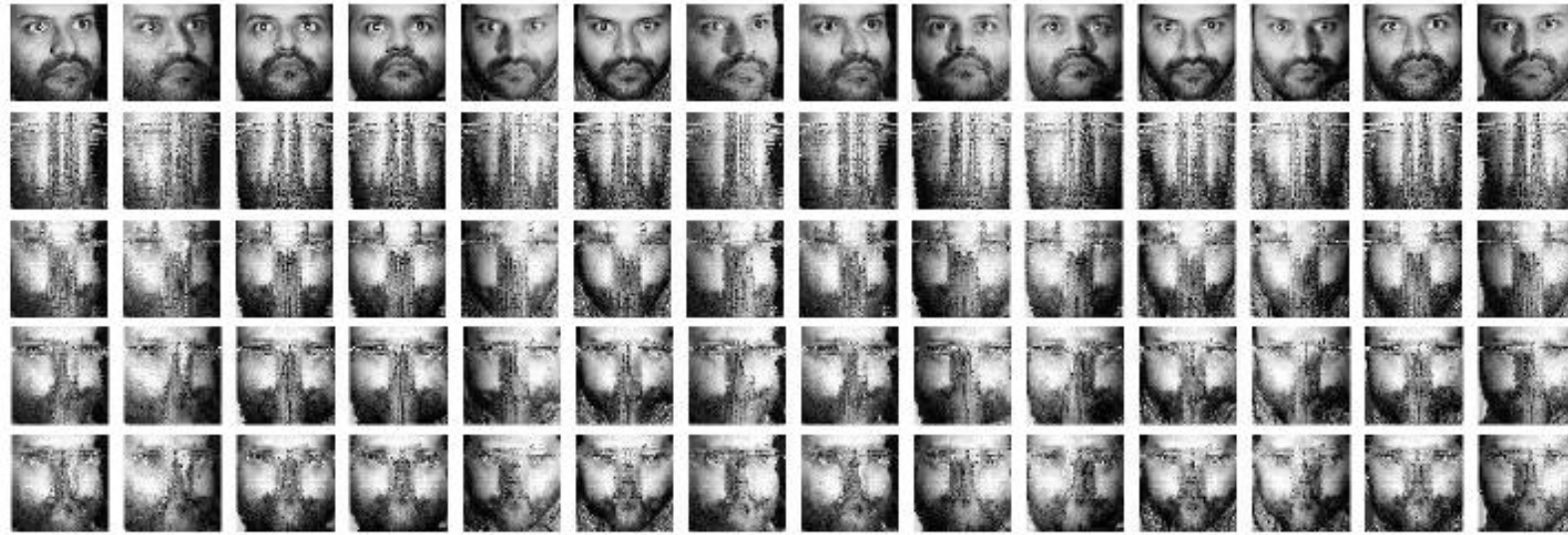


Figure 3. Comparison of fourteen element-rearranged images from the CMU PIE database based on the GLRAM-ER algorithm. The top row displays the original images. The subsequent rows show the rearranged images for  $d = 2, 3, 4$ , and  $5$ , respectively.

# Reconstruction Visualization



Figure 5. Comparison of seven reconstructed images from the FERET database. The top row displays the original images. The subsequent rows show reconstructed results from 2DPCA, 2DPCA-ER, GLRAM, and GLRAM-ER, respectively.

# Reconstruction Visualization



Figure 6. Comparison of fourteen reconstructed images from the CMU PIE database. The top row displays the original images. The subsequent rows show reconstructed results from 2DPCA, 2DPCA-ER, GLRAM, and GLRAM-ER, respectively.

# Classification Accuracy

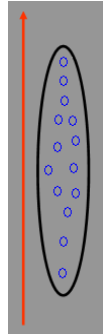
| Algorithm | FERET | CMU PIE |
|-----------|-------|---------|
| PCA       | 45.1  | 43.3    |
| LDA       | 91.8  | 76.2    |
| MFA       | 92.1  | 77.3    |
| 2DLDA     | 94.8  | 81.9    |
| 2DMFA     | 94.0  | 82.8    |
| 2DLDA-PA  | 95.6  | 83.7    |
| 2DMFA-PA  | 96.5  | 86.4    |

# Outline

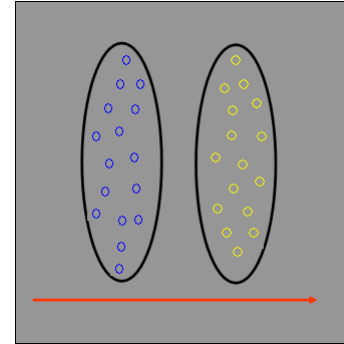
- ✓ Manifold learning
- ✓ Dimensionality Reduction for Tensor-based Objects
- ✓ Graph Embedding: A General Framework for Dimensionality Reduction



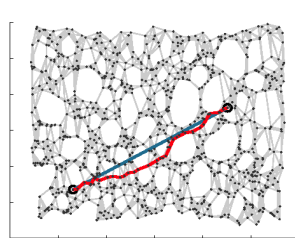
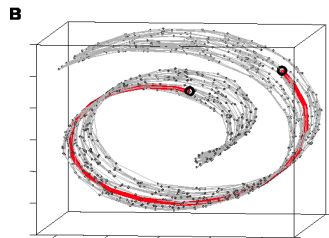
# Representative Previous Work



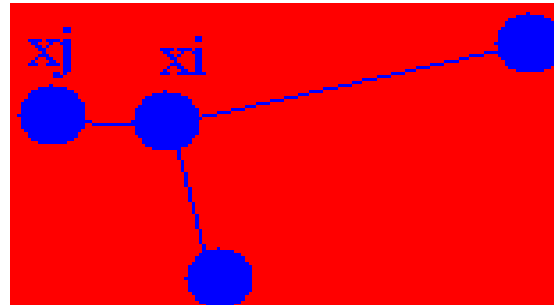
PCA



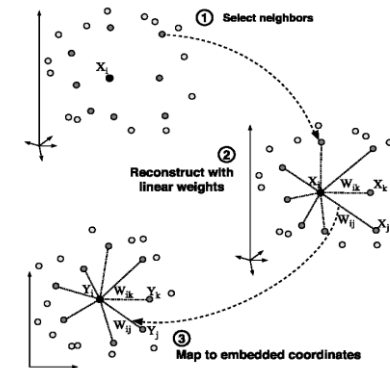
LDA



ISOMAP: Geodesic  
Distance Preserving  
J. Tenenbaum et al., 2000

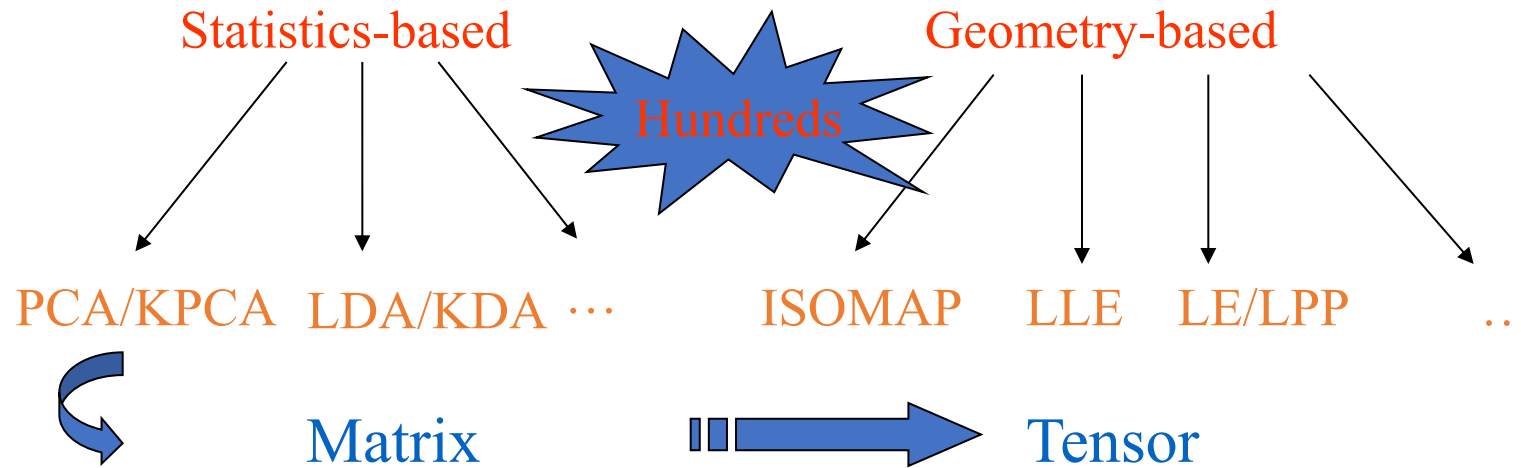


LE/LPP: Local Similarity  
Preserving, M. Belkin, P.  
Niyogi et al., 2001, 2003



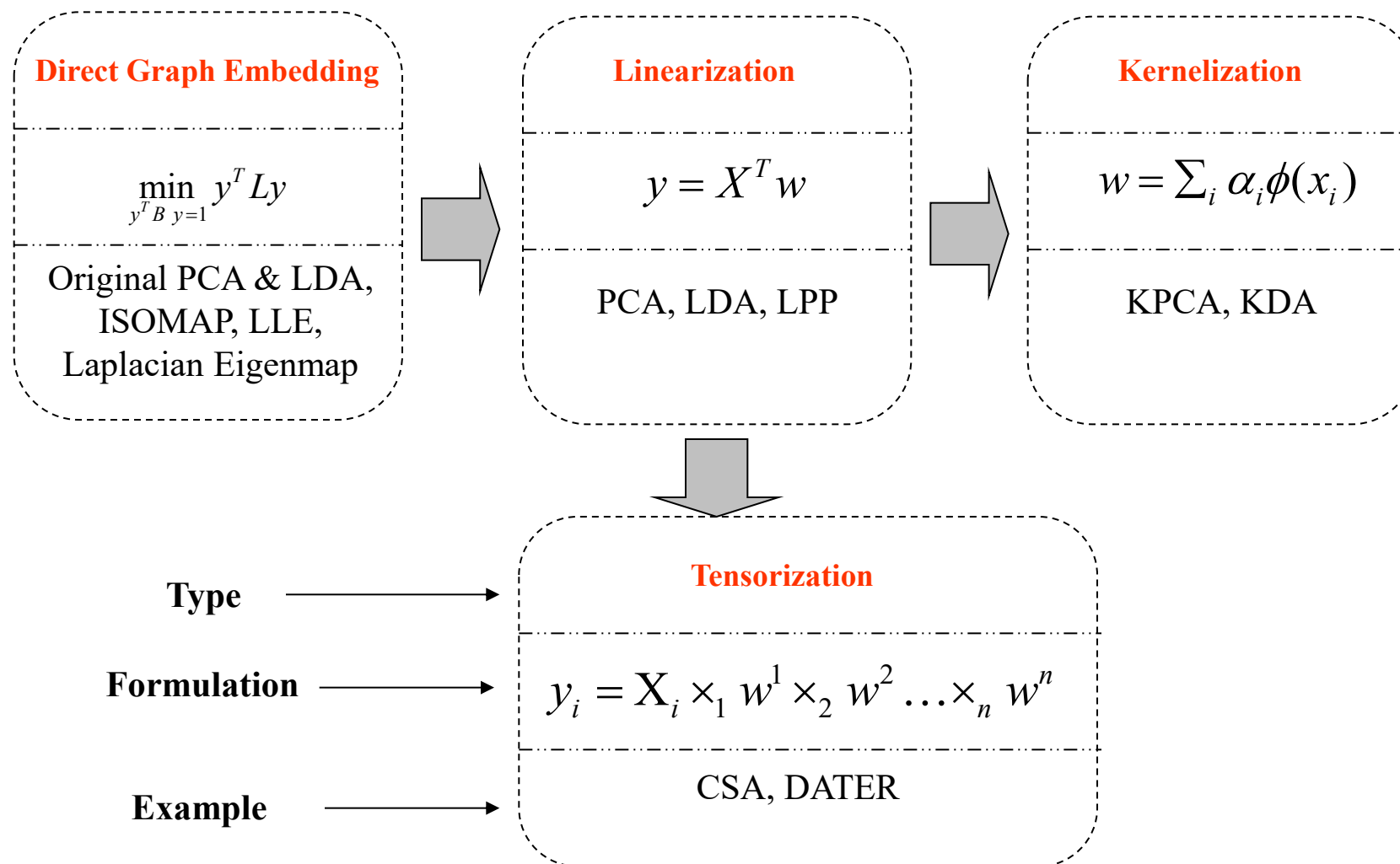
LLE: Local Neighborhood  
Relationship Preserving  
S. Roweis & L. Saul, 2000

# Dimensionality Reduction Algorithms



- Any *common perspective* to understand and explain these dimensionality reduction algorithms? Or any *unified formulation* that is shared by them?
- Any *general tool* to guide developing new algorithms for dimensionality reduction?

# Our Answers

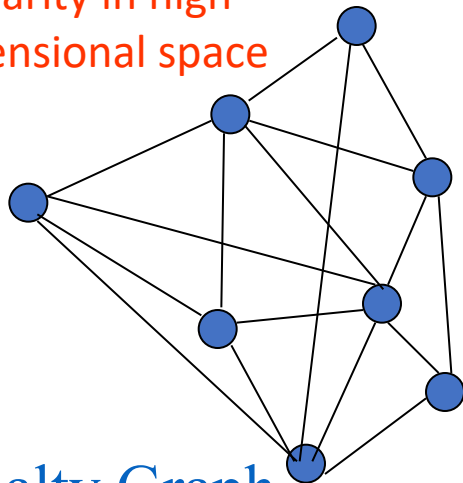


# Direct Graph Embedding

Intrinsic Graph:

$$G = [x_i, S_{ij}]$$

Similarity in high dimensional space



Penalty Graph

$$G^P = [x_i, S_{ij}^P]$$

$S, S^P$ : Similarity matrix (graph edge)

$L, B$ : Laplacian matrix from  $S, S^P$ ;

$$L = D - S, \quad D_{ii} = \sum_{j \neq i} S_{ij} \quad \forall i$$

Data in high-dimensional space and low-dimensional space (assumed as 1D space here):

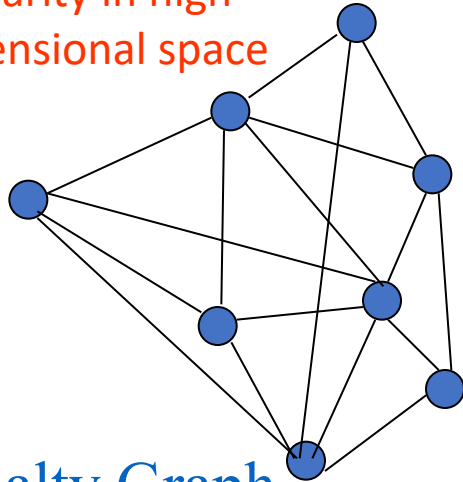
$$X = [x_1, x_2, \dots, x_N] \quad y = [y_1, y_2, \dots, y_N]^T$$

# Direct Graph Embedding -- Continued

Intrinsic Graph:

$$G = [x_i, S_{ij}]$$

Similarity in high dimensional space



Penalty Graph

$$G^P = [x_i, S_{ij}^P]$$

$S, S^P$ : Similarity matrix (graph edge)

$L, B$ : Laplacian matrix from  $S, S^P$ ;

$$L = D - S, \quad D_{ii} = \sum_{j \neq i} S_{ij} \quad \forall i$$

Data in high-dimensional space and low-dimensional space (assumed as 1D space here):

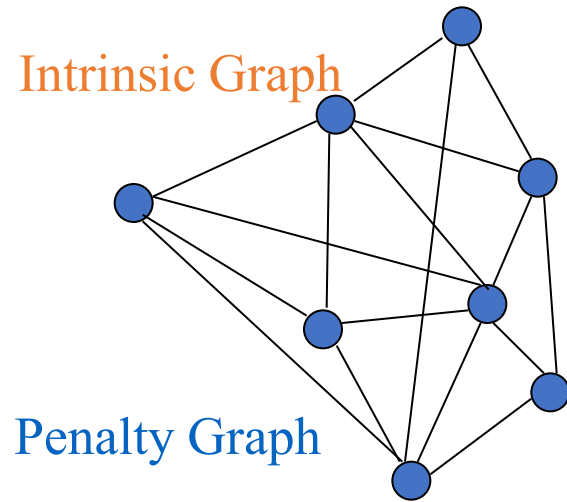
$$X = [x_1, x_2, \dots, x_N] \quad y = [y_1, y_2, \dots, y_N]^T$$

Criterion to Preserve Graph Similarity:

$$y^* \equiv \arg \min_{\substack{y^T y = 1 \text{ or} \\ y^T B y = 1 \text{ or} \\ y^T B y = 1}} \sum_{i \neq j} \|y_i - y_j\|^2 S_{ij} \quad \arg \min_{\substack{y^T y = 1 \text{ or} \\ y^T B y = 1}} y^T L y$$

Problem: It cannot handle new test data.

# Linearization



*Linear mapping function*

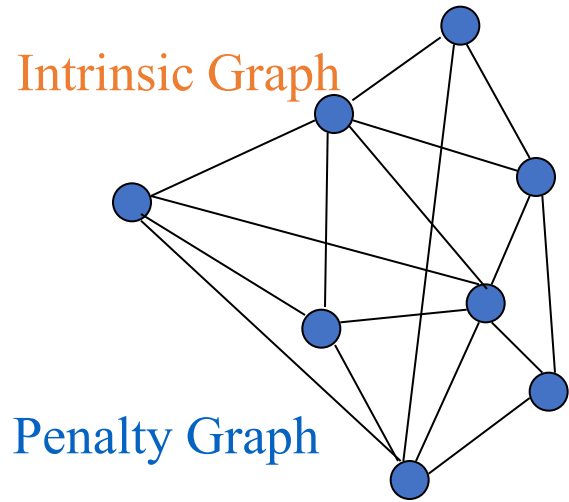
$$y = X^T w$$

## Objective function in Linearization

$$w^* = \arg \min_{\substack{w^T w = 1 \text{ or} \\ w^T X B X^T w = 1}} w^T X L X^T w$$

Problem: linear mapping function is not enough to preserve the real nonlinear structure?

# Kernelization



**Nonlinear mapping:**  $\phi: x_i \rightarrow \mathcal{F}$

*the original input space to another  
higher dimensional Hilbert space.*

**Constraint:**  $w = \sum_i \alpha_i \phi(x_i)$

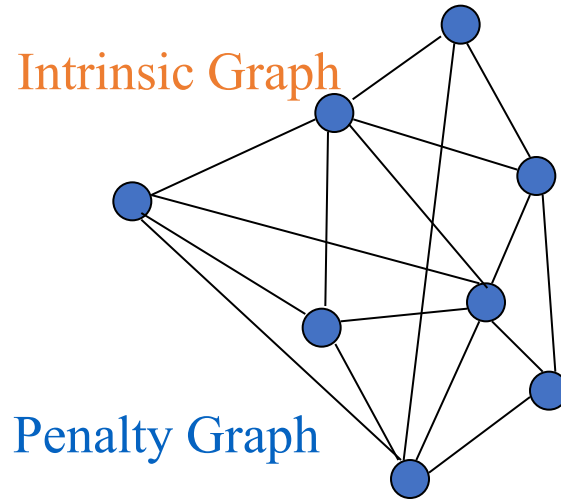
**Kernel matrix:**  $K_{ij} = k(x_i, x_j)$   $k(x, y) = \phi(x) \cdot \phi(y)$

**Objective function in Kernelization**

$$a^* = \arg \min_{\alpha} \alpha^T K L K \alpha$$

$\alpha^T K \alpha = 1$  or  
 $\alpha^T K B K \alpha = 1$

# Tensorization



Low dimensional representation is obtained as:

$$y_i = \mathbf{X}_i \times_1 w^1 \times_2 w^2 \dots \times_n w^n$$

## Objective function in Tensorization

$$(w^1, \dots, w^n)^* = \arg \min_{f(w^1, \dots, w^n)=1} \sum_{i \neq j} \| \mathbf{X}_i \times_1 w^1 \times_2 w^2 \dots \times_n w^n - \mathbf{X}_j \times_1 w^1 \times_2 w^2 \dots \times_n w^n \|^2 S_{ij}$$

where

$$f(w^1, \dots, w^n) = \sum_{i=1}^N \| \mathbf{X}_i \times_1 w^1 \times_2 w^2 \dots \times_n w^n \|^2 B_{ii} \quad \text{or}$$

$$f(w^1, \dots, w^n) = \sum_{i \neq j} \| \mathbf{X}_i \times_1 w^1 \times_2 w^2 \dots \times_n w^n - \mathbf{X}_j \times_1 w^1 \times_2 w^2 \dots \times_n w^n \|^2 S_{ij}^P$$



# Common Formulation



**Direct Graph Embedding**  $y^* = \arg \min_{\substack{y^T y = 1 \text{ or} \\ y^T B y = 1}} y^T L y$

**Linearization**  $w^* = \arg \min_{\substack{w^T w = 1 \text{ or} \\ w^T X B X^T w = 1}} w^T X L X^T w$

**Kernelization**  $a^* = \arg \min_{\substack{\alpha^T K \alpha = 1 \text{ or} \\ \alpha^T K B K \alpha = 1}} \alpha^T K L K \alpha$

**Tensorization**

$$(w^1, \dots, w^n)^* = \arg \min_{f(w^1, \dots, w^n) = 1} \sum_{i \neq j} \| \mathbf{X}_i \times_1 w^1 \times_2 w^2 \dots \times_n w^n - \mathbf{X}_j \times_1 w^1 \times_2 w^2 \dots \times_n w^n \|^2 S_{ij}$$

where  $f(w^1, \dots, w^n) = \sum_{i=1}^N \| \mathbf{X}_i \times_1 w^1 \times_2 w^2 \dots \times_n w^n \|^2 B_{ii}$  or  $f(w^1, \dots, w^n) = \sum_{i \neq j} \| \mathbf{X}_i \times_1 w^1 \times_2 w^2 \dots \times_n w^n - \mathbf{X}_j \times_1 w^1 \times_2 w^2 \dots \times_n w^n \|^2 S_{ij}^P$

# A General Framework for Dimensionality Reduction

| Algorithm     | $S$ & $B$ Definition                                                                | Embedding Type |
|---------------|-------------------------------------------------------------------------------------|----------------|
| PCA/KPCA/CSA  | $S_{ij} = 1/N, i \neq j; B = I$                                                     | L/K/T          |
| LDA/KDA/DATER | $S_{ij} = \delta_{i,l_j} / n_{l_i}, B = I - \frac{1}{N} ee^T$                       | L/K/T          |
| ISOMAP        | $S_{ij} = \tau(D_G)_{ij}, i \neq j; B = I$                                          | D              |
| LLE           | $S = M + M^T - M^T M; B = I$                                                        | D              |
| LE/LPP        | $S_{ij} = \exp \{-\ x_i - x_j\ ^2 / t\}$<br>$if \ x_i - x_j\  < \varepsilon; B = D$ | D/L            |

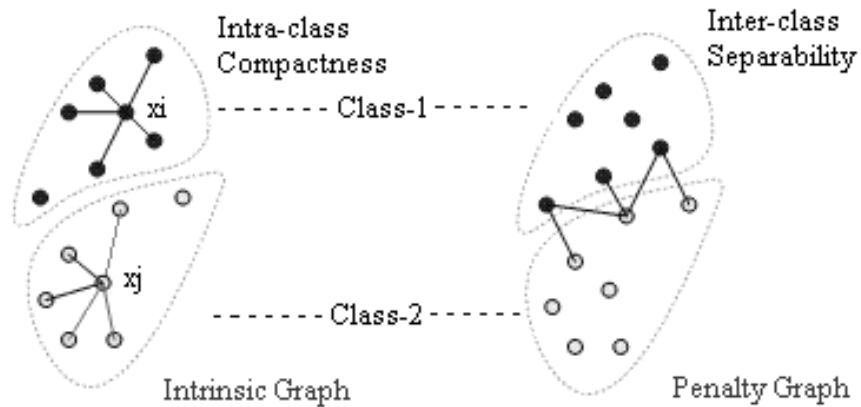
**D:** Direct Graph Embedding

**L:** Linearization

**K:** Kernelization

**T:** Tensorization

# New Dimensionality Reduction Algorithm: *Marginal Fisher Analysis*



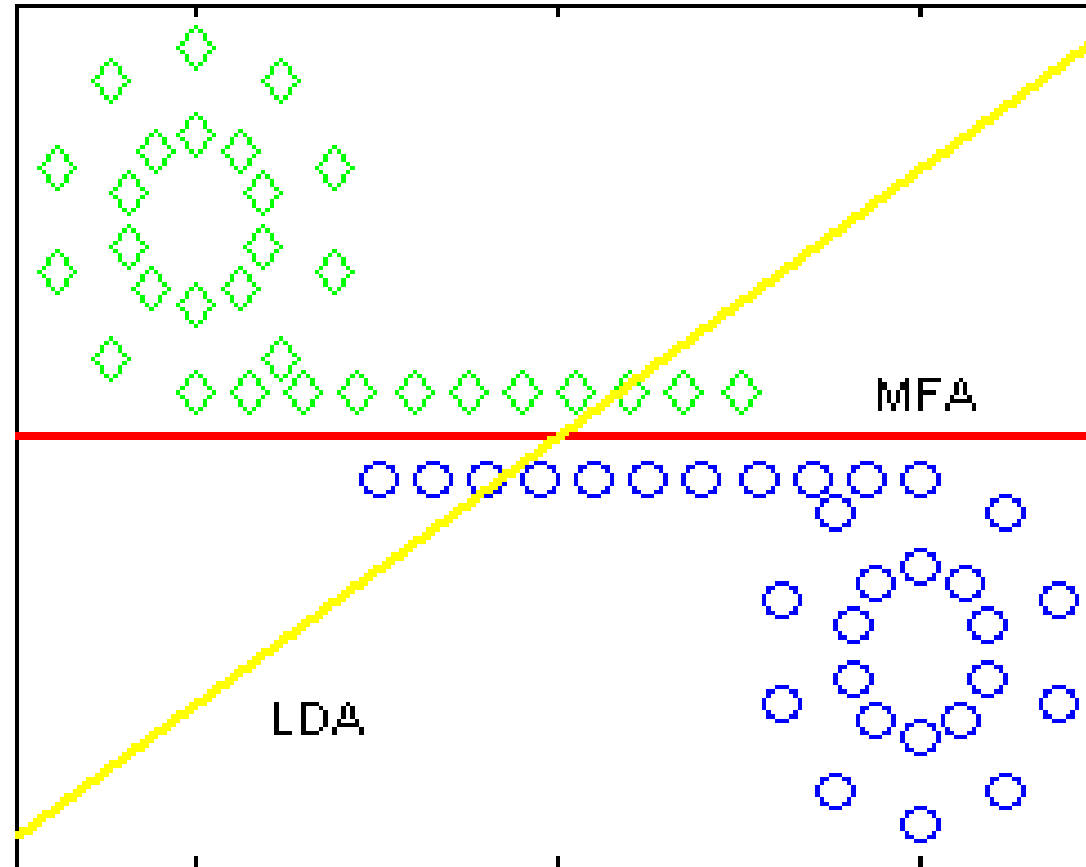
**Important Information  
for face recognition:**

- 1) Label information
- 2) Local manifold structure  
(*neighborhood* or *margin*)

$S_{ij} = 1$ : *if*  $x_i$  is among the  $k_1$ -nearest neighbors of  $x_j$  in the same class;  
 $0$ : *otherwise*

$S_{ij}^P = 1$ : *if* the pair  $(i,j)$  is among the  $k_2$  shortest pairs among the data set;  
 $0$ : *otherwise*

## *Marginal Fisher Analysis: Advantage*



No Gaussian distribution assumption

# Experiments: *Face Recognition*

| ORL                              | G3/P7        | G4/P6        |
|----------------------------------|--------------|--------------|
| PCA+LDA ( <i>Linearization</i> ) | 87.9%        | 88.3%        |
| <b>PCA+MFA (Ours)</b>            | <b>89.3%</b> | <b>91.3%</b> |
| KDA ( <i>Kernelization</i> )     | 87.5%        | 91.7%        |
| <b>KMFA (Ours)</b>               | <b>88.6%</b> | <b>93.8%</b> |
| DATER-2 ( <i>Tensorization</i> ) | 89.3%        | 92.0%        |
| <b>TMFA-2 (Ours)</b>             | <b>95.0%</b> | <b>96.3%</b> |

| PIE-1                            | G3/P7        | G4/P6        |
|----------------------------------|--------------|--------------|
| PCA+LDA ( <i>Linearization</i> ) | 65.8%        | 80.2%        |
| <b>PCA+MFA (Ours)</b>            | <b>71.0%</b> | <b>84.9%</b> |
| KDA ( <i>Kernelization</i> )     | 70.0%        | 81.0%        |
| <b>KMFA (Ours)</b>               | <b>72.3%</b> | <b>85.2%</b> |
| DATER-2 ( <i>Tensorization</i> ) | 80.0%        | 82.3%        |
| <b>TMFA-2 (Ours)</b>             | <b>82.1%</b> | <b>85.2%</b> |

# Summary

- Optimization framework that unifies previous dimensionality reduction algorithms as special cases.
- A new dimensionality reduction algorithm: Marginal Fisher Analysis.

**Thank you!**